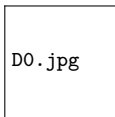


Lecture 11: Relational Algebra II

The Mathematics of Joins and Expression Trees

Z2004: Database Management Systems



IIT Madras Zanzibar

Tuesday, 24 March 2026

Recap & Learning Outcomes

Recap: Relational Algebra I

Yesterday, we learned the foundational, procedural operators of relational algebra:

- ▶ **Filter Rows:** Select (σ)
- ▶ **Filter Columns:** Project (π)
- ▶ **Set Ops:** Union ($\cup$), Difference ($-$)
- ▶ **Combine All:** Cartesian Product ($\times$)

Learning Outcomes

Today, we combine Cartesian Products ($\times$) and Selections (σ) to create the most powerful operations in databases: **JOINS**.

- ▶ **Natural Join:** Automatically match common columns.
- ▶ **Theta Join:** Join using custom conditions.
- ▶ **Outer Joins:** Keep data even when there is no match.
- ▶ **Expression Trees:** Visualize how the DBMS reads these queries.

What is Natural Join in SQL?

- ▶ **Overview:** Joins combine rows from two (or more) tables using related columns. A **NATURAL JOIN** is special because it implicitly matches columns with the same **name** (and compatible types).
- ▶ **Why use it?** It can make queries more concise when the intended matching columns already share the same names.
- ▶ **How it works:** SQL automatically finds common column names in both tables and applies an equality match on them, returning one copy of each matched column. Use with care to ensure those columns represent the correct relationship.

1. Natural Join ($\bowtie$)

The Smart Join

$R \bowtie S$ automatically joins two tables on all **common attributes** (columns with the exact same name).

It does this in 3 steps:

1. Compute Cartesian Product ($R \times S$)
2. Select rows where common attributes are equal
3. Project out duplicate columns

Mathematical Notation:

Student $\bowtie$ *Dept*

 Read as: "Student natural join Department."

Visualizing Natural Join

SID	Name	DeptID		DeptID	DName
S01	Amina	CS	$\bowtie$	CS	Comp Sci
S02	James	MA		MA	Math
S03	Priya	CS			



SID	Name	DeptID	DName
S01	Amina	CS	Comp Sci
S02	James	MA	Math
S03	Priya	CS	Comp Sci

✔ Notice that **DeptID** only appears once in the final result!

Natural Join ($\bowtie$): Example

Relation $r(A, B, C, D)$

A	B	C	D
α	1	α	a
β	2	γ	a
γ	4	β	b
α	1	γ	a
δ	2	β	b

Relation $s(B, D, E)$

B	D	E
1	a	α
3	a	β
1	a	γ
2	b	δ
3	b	ϵ



$r \bowtie s$ (joins on common attributes B and D)

A	B	C	D	E
α	1	α	a	α
α	1	α	a	γ
α	1	γ	a	α
α	1	γ	a	γ
δ	2	β	b	δ

2. Theta Join ($\bowtie_{\theta}$) & Equijoin

The Custom Join

What if columns have different names? Or what if we want to join based on a "greater than" ($>$) condition instead of equality? We use the **Theta Join**.

$$R \bowtie_{\theta} S \equiv \sigma_{\theta}(R \times S)$$

 **Read as:** "*R theta join S on condition theta.*"

Example (Non-Equality):

Find pairs of cities within 500 km:

$\text{City} \bowtie_{\text{distance} < 500} \rho_{\text{City2}}(\text{City})$

Equijoin vs. Natural Join

An **Equijoin** is a Theta Join where the condition *only* uses the equals sign ($=$).

THE DANGER OF NATURAL JOINS

Natural join blindly joins on ALL columns with the same name. If both 'Student' and 'Dept' have a column called 'Name' (Student Name vs. Dept Name), the Natural Join will try to match them and output nothing!

Solution: Always prefer explicit Equijoins!

$\text{Student} \bowtie_{\text{Student.DeptID}=\text{Dept.ID}} \text{Dept}$

Visualizing Equijoin & Theta Join

1. The Equijoin

Student ⋈_{Student.DeptID=Dept.ID} *Dept*

SID	DeptID	ID	DName
1	CS	CS	Sci
2	MA	MA	Math



SID	DeptID	ID	DName
1	CS	CS	Sci
2	MA	MA	Math

i Notice that the Equijoin keeps BOTH join columns (DeptID and ID), unlike the Natural Join!

2. Non-Equality Theta Join

Task ⋈_{Task.Budget > Equip.Cost} *Equip*

Task	Budg	Item	Cost
T1	\$500	PC	\$400
T2	\$200	Desk	\$150



Task	Budg	Item	Cost
T1	\$500	PC	\$400
T1	\$500	Desk	\$150
T2	\$200	Desk	\$150

✓ Every combination where the budget is strictly greater than the cost is returned.

Equijoin: Example

Customer

Cid	Cname	C_country
11	David	UK
12	Mary	US
13	John	China

Agent

Aid	Aname	A_country
101	Sara	China
102	Mike	Italy
103	Jim	UK

Customer ⋈_{*Customer.C_country=Agent.A_country*} *Agent*



Cid	Cname	C_country	Aid	Aname	A_country
11	David	UK	103	Jim	UK
13	John	China	101	Sara	China

Theta Join: Example (>)

Customer

Name	Country	CreditLimit
W. William	England	10000
B. Knight	USA	50000
L. Dove	USA	2000

Product

Product	Type	Price
Sport-100 Helmet, Red	Bikes	2500
Men's Sports Shorts, M	Clothing	12000

Customer ⋈_{*Customer.CreditLimit > Product.Price*} *Product*



Name	Country	CreditLimit	Product	Type	Price
W. William	England	10000	Sport-100 Helmet, Red	Bikes	2500
B. Knight	USA	50000	Sport-100 Helmet, Red	Bikes	2500
B. Knight	USA	50000	Men's Sports Shorts, M	Clothing	12000

3. Outer Joins: Preserving Unmatched Data

The Missing Data Problem

Standard joins (Inner Joins) **drop** any row that does not have a matching partner. If a student hasn't registered for classes yet, they disappear from the result!

Outer Joins preserve these rows by filling in the missing data with **NULL**.

- ▶ **Left** ($\bowtie$): Keep all left rows.
- ▶ **Right** ($\bowtie$): Keep all right rows.
- ▶ **Full** ($\Join$): Keep rows from both.

Left Outer Join Visualized

SID	Name		SID	Course
S01	Amina	$\bowtie$	S01	C101
S02	James		S01	C102
S03	Priya		S03	C101



SID	Name	Course
S01	Amina	C101
S01	Amina	C102
S02	James	NULL
S03	Priya	C101

James is preserved in the output, even though he takes no courses!

Visualizing Right & Full Outer Joins

Right Outer Join ($\bowtie$)

Keeps all rows from the **Right** table. If the left table has no match, fill with NULLs.

Student $\bowtie$ *Course*

SID	Name		SID	C_ID
S01	Amina	$\bowtie$	S01	C101
			S99	C999



SID	Name	C_ID
S01	Amina	C101
S99	NULL	C999

Course C999 has student S99, but S99 doesn't exist in the Student table! The right side is preserved.

Full Outer Join ($\ltimes$)

Keeps ALL rows from **Both** tables. Fills missing matches with NULLs on either side.

Student $\ltimes$ *Course*

SID	Name		SID	C_ID
S01	Amina	$\ltimes$	S01	C101
S02	James		S99	C999



SID	Name	C_ID
S01	Amina	C101
S02	James	NULL
S99	NULL	C999

Both James (no courses) and S99 (no name) are

4. Expression Trees: Visualizing Query Plans

Inside the Optimizer

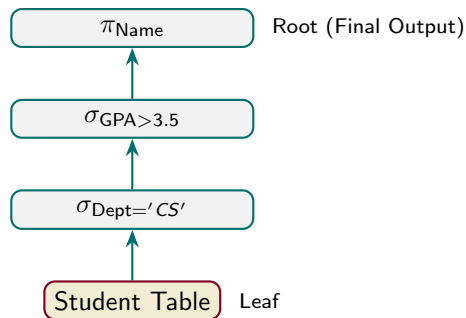
When you send a query, the DBMS maps it into an **Expression Tree** to figure out how to execute it efficiently. Data flows **bottom-up**.

Tree Rules:

- ▶ **Leaves:** Base Tables.
- ▶ **Internal Nodes:** Operations (σ , π , $\bowtie$).
- ▶ **Root:** The final output.

Query: “Get the Names of CS students with a GPA > 3.5”

$\pi_{\text{Name}}(\sigma_{\text{GPA} > 3.5}(\sigma_{\text{Dept} = \text{'CS'}}(\text{Student})))$



Class Activity: The Join Challenge

The Database

Employee Table

E_ID	Name	DeptID
1	Alice	D1
2	Bob	D2
3	Carol	NULL

Department Table

DeptID	DName
D1	Sales
D2	HR
D3	IT



Instructions:

Take 5 minutes to solve the 3 tasks on the right.

Tasks

Task 1: The Natural Join

- ▶ Draw the resulting table for:
Employee $\bowtie$ Department

▶ *Hint: Does Carol survive? Does IT survive?*

Task 2: The Left Outer Join

- ▶ Draw the resulting table for:
Employee $\ltimes$ Department

Task 3: Expression Tree

- ▶ Draw the Expression Tree for this query: Get the **Names** of Employees working in **Sales**.

- ▶ *Algebra:*

$\pi_{\text{Name}}(\sigma_{\text{DName}='Sales'}(\text{Emp} \bowtie \text{Dept}))$

Class Activity: Solutions

Task 1 & 2 Solutions

Task 1 Result: Natural Join ($\bowtie$)

E_ID	Name	DeptID	DName
1	Alice	D1	Sales
2	Bob	D2	HR

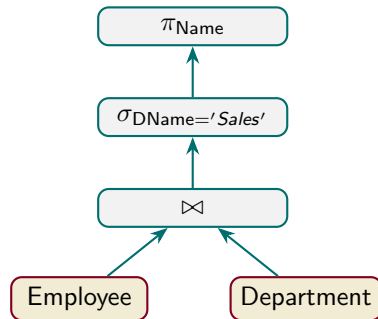
Carol (NULL Dept) and IT (No Employees) are dropped!

Task 2 Result: Left Outer ($\ltimes$)

E_ID	Name	DeptID	DName
1	Alice	D1	Sales
2	Bob	D2	HR
3	Carol	NULL	NULL

Carol is preserved because she is in the Left table (Employee).

Task 3 Solution: Expression Tree



✓ Notice how the two base tables form the leaves, merge at the Join node, and filter upwards to the root!

Lecture Summary & Next Steps

Key Takeaways

- ✓ **Natural Join ($\bowtie$):** Joins on common attributes and eliminates duplicate columns automatically.
- ✓ **Theta Join ($\bowtie_{\theta}$):** Allows you to join using any custom predicate (e.g., $<$, $>$, $=$).
- ✓ **Inner vs Outer:** Inner joins drop unmatched tuples. Outer joins ($\ltimes$, $\ltimes$) preserve them using NULL values.
- ✓ **Expression Trees:** Visual representations showing how operations are chained from the tables (leaves) up to the final result (root).

Next Lecture (L12)

Relational Algebra III: Division & Equivalence

- ➔ **Division ($\div$):** How to answer “for all” queries mathematically.
- ➔ **Equivalence Rules:** How the Expression Tree reorganizes itself to make queries faster.

Reading Assignment

- 📖 Read **Silberschatz, Chapter 6** (Sections 6.1–6.3) to prepare for Lecture 12.